

```

1  """
2  -----
3  File: DatabaseVault.py
4
5  Summary: Ingests raw IPUMS data into a strictly normalized SQLite database.
6           Reads the CSV sequentially, buffering households by SERIAL.
7           Maps relationships using POPLOC/MOMLOC to extract exact HISTIDs
8           for parents, and writes to 'families' and 'individuals' tables.
9
10  Design:  Single-threaded, sequential read.
11           Assigns "Future Bosselstink" to nameless records.
12           Saves the entire raw CSV row as JSON "bread crumbs" so no data is lost.
13  -----
14  """
15
16  import argparse
17  import csv
18  import datetime
19  import json
20  import os
21  import sqlite3
22  import time
23  import sys
24
25  # Add the 'python' directory and project root to sys.path so we can import properly
26  script_dir = os.path.dirname(os.path.abspath(__file__))
27  project_root = os.path.abspath(os.path.join(script_dir, '..'))
28  for p in [script_dir, project_root]:
29      if p not in sys.path:
30          sys.path.append(p)
31
32  from utils import gen_logging
33
34  # =====
35  # TUNING KNOBS
36  # =====
37  BATCH_SIZE = 5_000 # Number of *Households* to buffer before committing to DB
38  MASTER_DB = r"D:\Data\Genealogy_Data\MasterVault_Relational.db"
39  input_directory = r"C:\tempc\ShortTermCSVfiles"
40
41  # =====
42  # DATABASE SETUP
43  # =====
44  def setup_database(db_path, logger):
45      logger.info(f"Connecting to database (Setup): {db_path}")
46      os.makedirs(os.path.dirname(db_path), exist_ok=True)
47      conn = sqlite3.connect(db_path)
48      cursor = conn.cursor()
49
50      # DECISION: Write-Ahead Logging (WAL) ensures that if the script is killed (Ctrl+C)
51      # mid-run,
52      # the database does not corrupt. It safely rolls back incomplete transactions and
53      # allows parallel reads.
54      cursor.execute("PRAGMA journal_mode=WAL")
55
56      # DECISION: Enforce Relational Integrity. SQLite defaults to OFF. Turning this ON
57      # ensures
58      # we cannot insert an individual into a family_id that doesn't exist, preventing
59      # orphaned records.
60      cursor.execute("PRAGMA foreign_keys = ON;")
61
62      # For clean testing, drop the tables so every run is fresh
63      cursor.execute("DROP TABLE IF EXISTS individuals;")
64      cursor.execute("DROP TABLE IF EXISTS families;")
65
66      # DECISION: The Families Table represents the "Nuclear Family Unit", NOT just the

```

```

physical house.
63 # By tracking 'famunit', we separate multiple families living under the same roof
    (e.g., boarders, servants).
64 # We also extract core metrics (head_histid, spouse_histid, numprec) up to the
    family level
65 # to make downstream querying incredibly fast without needing complex JOINS.
66 cursor.execute('''
67     CREATE TABLE IF NOT EXISTS families (
68         family_id TEXT PRIMARY KEY,
69         year INTEGER,
70         serial TEXT,
71         famunit TEXT,
72         head_histid TEXT,
73         spouse_histid TEXT,
74         hhstype TEXT,
75         numprec TEXT,
76         pernum TEXT,
77         eldch TEXT,
78         yngch TEXT,
79         relate TEXT
80     )
81 ''')
82
83 # DECISION: The Individuals Table stores the raw demographics and links to the
    Family Table via Foreign Key.
84 # The 'raw_data' column acts as our "Bread Crumbs", storing the entire JSON row so
    we
85 # never lose any of the 50+ IPUMS variables, even if we don't explicitly parse them
    into columns today.
86 # HISTID is used as the Primary Key because IPUMS guarantees it is a universally
    unique, permanent ID.
87 cursor.execute('''
88     CREATE TABLE IF NOT EXISTS individuals (
89         histid TEXT PRIMARY KEY,
90         first_name TEXT,
91         last_name TEXT,
92         year INTEGER,
93         sample TEXT,
94         serial TEXT,
95         pernum TEXT,
96         famunit TEXT,
97         age INTEGER,
98         sex TEXT,
99         birthyr INTEGER,
100        bpld TEXT,
101        fbpl TEXT,
102        mbpl TEXT,
103        father_histid TEXT,
104        mother_histid TEXT,
105        family_id TEXT,
106        raw_data TEXT, -- The Bread Crumbs! (JSON)
107        FOREIGN KEY (family_id) REFERENCES families(family_id)
108    )
109 ''')
110
111 conn.commit()
112 logger.info(f"Disconnecting from database (Setup): {db_path}")
113 conn.close()
114 logger.info(f"Database ready: {db_path}")
115
116 # =====
117 # HOUSEHOLD PROCESSOR
118 # =====
119 def process_household(rows):
120     """
121     Takes a buffer of rows for a single SERIAL number.

```

```

122 Maps children to their exact parents using HISTID, and constructs the tables.
123 """
124 # DECISION: Create a quick lookup dictionary mapping the enumerator's line number
125 # (PERNUM)
126 # to the person's permanent IPUMS UUID (HISTID). This allows us to instantly resolve
127 # POPLOC/MOMLOC pointers.
128 pernum_to_histid = {str(r.get('PERNUM', '')).strip(): str(r.get('HISTID', '')).strip()
129                     for r in rows}
130
131 families_dict = {}
132 individuals_to_insert = []
133
134 for row in rows:
135     histid = str(row.get('HISTID', '')).strip()
136     year = str(row.get('YEAR', '')).strip()
137     serial = str(row.get('SERIAL', '')).strip()
138     pernum = str(row.get('PERNUM', '')).strip()
139     famunit = str(row.get('FAMUNIT', '1')).strip()
140
141     poploc = str(row.get('POPLOC', '0')).strip()
142     momloc = str(row.get('MOMLOC', '0')).strip()
143
144     # DECISION: Use POPLOC and MOMLOC to assign the exact parent HISTID directly to
145     # the child.
146     # This means the database natively understands the bloodline without needing
147     # future Python processing.
148     father_histid = pernum_to_histid.get(poploc) if poploc != '0' else None
149     mother_histid = pernum_to_histid.get(momloc) if momloc != '0' else None
150
151     # DECISION: Build a unique family ID using Year, Serial (House), and FamUnit
152     # (Nuclear Family).
153     family_id = f"{year}_{serial}_{famunit}"
154
155     # DECISION: Capture the Head of Household and Spouse directly into the family
156     # record as we iterate.
157     if family_id not in families_dict:
158         families_dict[family_id] = {
159             'year': year, 'serial': serial, 'famunit': famunit,
160             'head_histid': None, 'spouse_histid': None,
161             'hhtype': str(row.get('HHTYPE', '')).strip(),
162             'numprec': str(row.get('NUMPREC', '')).strip(),
163             'pernum': pernum,
164             'eldch': str(row.get('ELDCH', '')).strip(),
165             'yngch': str(row.get('YNGCH', '')).strip(),
166             'relate': str(row.get('RELATE', '')).strip()
167         }
168
169     related = str(row.get('RELATED', '')).zfill(4).lower()
170     if '0100' in related or 'head' in related:
171         families_dict[family_id]['head_histid'] = histid
172     elif '0200' in related or 'spouse' in related or 'wife' in related:
173         families_dict[family_id]['spouse_histid'] = histid
174
175     # DECISION: Never allow NULL or blank names. If the IPUMS 100% database lacks
176     # names,
177     # we assign a highly unique placeholder ("Future Bosselstink"). Downstream
178     # GEDCOM overlays
179     # will explicitly look for and overwrite these placeholders with true historical
180     # names.
181     first_name = str(row.get('NAMEFRST', '')).strip()
182     last_name = str(row.get('NAMELAST', '')).strip()
183
184     if not first_name:
185         first_name = "Future"
186     if not last_name:
187         last_name = "Bosselstink"

```

```

178
179     # DECISION: The JSON Bread Crumbs. Serialize the entire raw CSV row into a JSON
180     raw_data_json = json.dumps(row)
181
182     individuals_to_insert.append((
183         histid, first_name, last_name, year, row.get('SAMPLE'), serial, pernum,
184         famunit,
185         row.get('AGE'), row.get('SEX'), row.get('BIRTHYR'), row.get('BPLD'),
186         row.get('FBPL'), row.get('MBPL'),
187         father_histid, mother_histid, family_id,
188         raw_data_json
189     ))
190
191     families_to_insert = [
192         (fid, data['year'], data['serial'], data['famunit'], data['head_histid'], data[
193             'spouse_histid'],
194         data['hhtype'], data['numprec'], data['pernum'], data['eldch'], data['yngch'],
195         data['relate'])
196         for fid, data in families_dict.items()
197     ]
198
199     return individuals_to_insert, families_to_insert
200
201 # =====
202 # INGESTION LOOP
203 # =====
204 def ingest_to_vault(input_csv, db_path, logger, record_limit=None):
205     logger.info(f"Opening database for ingestion: {db_path}")
206     conn = sqlite3.connect(db_path)
207     conn.execute("PRAGMA journal_mode=WAL")
208     conn.execute("PRAGMA foreign_keys = ON;")
209     cursor = conn.cursor()
210
211     ind_insert_query = """
212         INSERT OR IGNORE INTO individuals
213         (histid, first_name, last_name, year, sample, serial, pernum, famunit,
214         age, sex, birthyr, bpld, fbpl, mbpl, father_histid, mother_histid, family_id,
215         raw_data)
216         VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
217     """
218
219     fam_insert_query = """
220         INSERT OR IGNORE INTO families
221         (family_id, year, serial, famunit, head_histid, spouse_histid, hhtype, numprec,
222         pernum, eldch, yngch, relate)
223         VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
224     """
225
226     ind_batch = []
227     fam_batch = []
228     count = 0
229     households_processed = 0
230     start_time = time.time()
231
232     logger.info(f"Opening CSV file for sequential read: {input_csv}")
233     with open(input_csv, mode='r', encoding='utf-8', errors='replace') as infile:
234         reader = csv.DictReader(infile, delimiter=',')
235         current_serial = None
236         household_buffer = []
237
238         for row in reader:
239             raw_serial = str(row.get('SERIAL', '')).strip()
240
241             # Initialize on the very first row
242             if current_serial is None:

```

```

238         current_serial = raw_serial
239
240     # DECISION: Sequential Household Buffering. The CSV is historically ordered
241     # by the census taker.
242     # Everyone in the same house (SERIAL) is listed sequentially. We buffer rows
243     # in memory until
244     # the SERIAL changes, meaning we have the complete house and can process
245     # them as a single atomic unit.
246     if raw_serial != current_serial:
247         inds, fams = process_household(household_buffer)
248         ind_batch.extend(inds)
249         fam_batch.extend(fams)
250
251         households_processed += 1
252         count += len(household_buffer)
253
254         # Reset the buffer for the new household
255         household_buffer = [row]
256         current_serial = raw_serial
257
258     # DECISION: Graceful Shutdown. By checking the limit here and clearing
259     # the buffer,
260     # we ensure we don't commit a partially-read family if the limit is
261     # reached mid-household.
262     if record_limit and count >= record_limit:
263         logger.info(f" -> Reached record limit ({record_limit:,}). Stopping
264         early for review.")
265         household_buffer = [] # Clear the buffer so we don't insert a
266         partial household
267         break
268     else:
269         # Still in the same household, add to buffer
270         household_buffer.append(row)
271
272     # DECISION: Batch commits prevent the script from blowing up RAM on massive
273     # 100GB files.
274     if len(fam_batch) >= BATCH_SIZE:
275         # DECISION: Foreign Key constraints require that parent records
276         # (families) are inserted
277         # before child records (individuals) pointing to them.
278         cursor.executemany(fam_insert_query, fam_batch)
279         cursor.executemany(ind_insert_query, ind_batch)
280         conn.commit()
281
282         logger.info(f" -> Processed {households_processed:,} households ({count
283         :,} individuals)...")
284         ind_batch = []
285         fam_batch = []
286
287     # Catch the final household buffer when the file ends
288     if household_buffer:
289         inds, fams = process_household(household_buffer)
290         fam_batch.extend(fams)
291         ind_batch.extend(inds)
292         count += len(household_buffer)
293         households_processed += 1
294
295     if fam_batch:
296         cursor.executemany(fam_insert_query, fam_batch)
297         cursor.executemany(ind_insert_query, ind_batch)
298         conn.commit()
299
300 conn.close()
301 elapsed = round((time.time() - start_time) / 60, 2)
302 logger.info(f" [{os.path.basename(input_csv)}] DONE - {count:,} records in {elapsed
303 } min.")

```

```

293
294
295 # =====
296 # MAIN
297 # =====
298 if __name__ == '__main__':
299     parser = argparse.ArgumentParser(description="Ingest census CSVs into a relational
300     SQLite vault.")
301     parser.add_argument("--limit", type=int, default=100_000, help="Stop reading after
302     this many individuals (0 for all).")
303     args = parser.parse_args()
304
305     record_limit = args.limit if args.limit > 0 else None
306
307     main_logger = gen_logging.setup_logging(logger_name="MAIN")
308
309     main_logger.info("=====")
310     main_logger.info("  RELATIONAL DATABASE INGESTION (SINGLE THREADED)")
311     main_logger.info("=====")
312
313     setup_database(MASTER_DB, main_logger)
314
315     csv_files = [f for f in os.listdir(input_directory) if f.endswith(".csv")]
316
317     for filename in csv_files:
318         file_path = os.path.join(input_directory, filename)
319         ingest_to_vault(file_path, MASTER_DB, main_logger, record_limit)
320
321     main_logger.info("\nAll CSV files have been processed and loaded into the Relational
322     Vault.")

```